

25. La boucle du jeu : while (true)

Tout jeu est une boucle : lire, calculer, dessiner, attendre — soixante fois par seconde.

1. Ce qu'il faut comprendre

Une **boucle** répète un bloc de code. `while (condition) { ... }` le répète **tant que** la condition est vraie. `while (true)` ne s'arrête **jamais** : c'est voulu. Une console n'a pas de « fin de programme » — si `main` se terminait, le processeur partirait lire n'importe quoi.

Chaque passage dans le bloc s'appelle un **tour** (on dit aussi une **itération**). `image()` attend que l'écran ait fini de se dessiner : un tour dure donc exactement **une image**, soit 1/60 de seconde. C'est l'**horloge** du jeu.

Tous les jeux Game Boy ont cette forme : **lire les touches** → **faire évoluer le monde** → **dessiner** → **attendre l'image suivante**, et recommencer. Tetris, Zelda, Pokémon : même boucle, plus de choses dedans.

Ici, on s'en sert pour faire une **montre**. `imagesVues` compte les tours ; quand il atteint 60, une seconde est passée : on le remet à zéro et on ajoute une seconde. C'est ainsi que l'on compte plus loin que 255 — avec **deux** variables, comme les heures et les minutes.

Remarque qu'on n'écrit les secondes à l'écran **qu'au moment où elles changent**, une fois par seconde. Écrire à l'écran n'est pas gratuit : chaque lettre doit attendre un moment précis, une courte pause de la console. On n'écrit donc que ce qui change. La leçon « L'écran ne s'écrit que pendant le VBlank », un peu plus loin, explique pourquoi.

À toi : ajoute les minutes : quand `secondes` atteint 60, remets-le à 0 et augmente `minutes`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **582 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	64 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	272 o	← CopierTuiles — 17 lettres sur 43
"SECONDES:"	9 o	← texte() — un texte gravé

Lettres gravées : CDENOS0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

nombre()	× 2	20 o sur place, dans main()	+ EcrireNombre (33 o)
texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : compte les secondes qui passent, grâce à la boucle du jeu.
//
// Ce qui est important : while (true) ... image(); est LA boucle de tout jeu.
// La console montre 60 images par seconde ; image() attend la suivante.
// Donc un tour de boucle = une image = 1/60 de seconde.

#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t imagesVues = 0;    // Combien d'images depuis la dernière seconde (0 à 59).
    uint8_t secondes = 0;     // Combien de secondes en tout.

    texte(1, 5, "SECONDES:"); // Le titre...
    nombre(12, 5, secondes);  // ... et la valeur de départ : 000.

    while (true) {
        image();              // un tour de boucle = une image

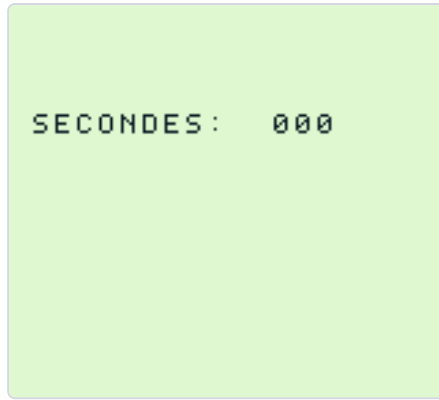
        imagesVues++;          // Une image de plus : + 1.
        if (imagesVues == 60) { // Arrivée à 60 ? (« == » compare ; un seul « = » rangerait)
            // Si oui, une seconde est passée :
            imagesVues = 0;      // on recommence à compter les images à 0,
            secondes++;          // on ajoute une seconde,
            nombre(12, 5, secondes); // et on l'affiche.
        }                      // Sinon (moins de 60), on ne fait rien de plus.
    }
}
```

3. Ce qui se passe, étape par étape

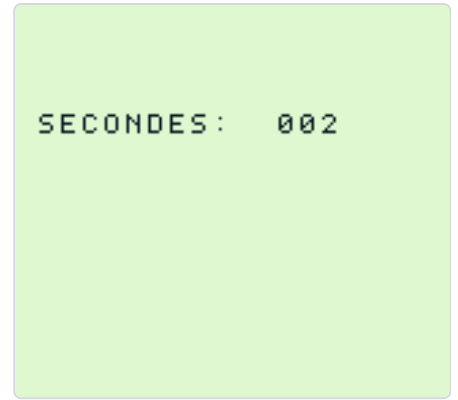
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



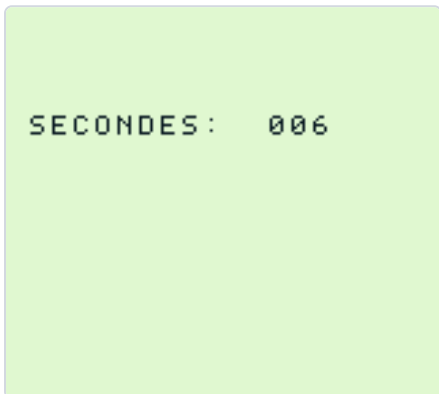
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Les secondes avancent d'une à chaque seconde : 001, 002, 003...

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ à peu près 120 images font 2 secondes de plus (0 puis 2)
- ✓ le compteur d'images ne dépasse jamais 59
- ✓ l'écran montre les secondes

Ensuite : **26. while avec une condition : la boucle qui s'arrête** — chapitre 5 : Les boucles : répéter. · gameboy3 — écrire une cartouche Game Boy en C++